

GPU 编程实验报告

2411567 陈涵

2026.04.02

目录

1 问题描述	3
1.1 期末研究报告宏观问题	3
1.2 本次 GPU 编程实验子问题	3
2 HIP 课程学习	3
2.1 Lab1 Vector Addition	3
2.1.1 学习内容	3
2.1.2 题目截图	3
2.2 Lab2 Matrix Transpose	6
2.2.1 学习内容	6
2.2.2 题目截图	7
2.3 Lab3 Histogram	8
2.3.1 学习内容	8
2.3.2 题目截图	9
2.4 Lab4 Reduction	10
2.4.1 学习内容	10
2.4.2 题目截图	11
2.5 Lab5 Monte Carlo Integration	12
2.5.1 学习内容	12
2.5.2 题目截图	13
2.6 Lab6 K-Means Clustering	14
2.6.1 学习内容	14
2.6.2 题目截图	15
3 指令生成加速 (CUDA)	17
3.1 实现思路	17
3.2 代码实现	18
4 实验结果与分析	21

5 总结**22**

1 问题描述

1.1 期末研究报告宏观问题

本课题旨在构建一套高效的混合并行口令猜测系统,需融合 MPI(跨节点任务分发)、OpenMP/Pthread 与 SIMD/GPU 等技术。核心研究问题在于:如何突破优先队列操作的锁竞争瓶颈、实现细粒度负载均衡、缓解 CPU-GPU/内存数据传输延迟,并量化分析“口令生成”与“哈希计算”在完整链路中的算力占比,从而设计最优的异构并行流水线。

1.2 本次 GPU 编程实验子问题

本次实验代码均存于此仓库的 lab5 中,并不尝试将整个 PCFG 模型完全迁移到 GPU,因为优先队列维护、PT 扩展、pivot 更新等操作具有较强的控制依赖和动态数据结构特征,直接 GPU 化会引入复杂的同步与内存管理问题,且未必能获得实际加速。因此,本实验采用 CPU-GPU 协同方式:CPU 负责模型训练、优先队列维护和 PT 调度,GPU 负责对每个 PT 的最后一个 segment 进行批量字符串拼接生成。

2 HIP 课程学习

2.1 Lab1 Vector Addition

2.1.1 学习内容

这个 Lab 通过向量加法示例介绍了 HIP 在 AMD GPU 上的基础编程模型,主要学习了 Kernel、Grid、Block 和 Thread 的层次结构,如何计算线程的全局索引并让每个线程处理一个数据元素,同时理解 AMD GPU 中 Compute Unit、Wavefront 和 SIMD 等硬件概念的对应关系;实验还分析了 block size、wavefront 对齐、occupancy 和内存带宽对性能的影响,帮助掌握 GPU 并程序的基本写法和性能优化思路。

2.1.2 题目截图

Exercise: Thread Index Calculation

Fill in the global index for each thread:

blockIdx.x	blockDim.x	threadIdx.x	Global Index
0	256	100	256
2	128	50	356
5	64	0	320

图 1: Exercise: Thread Index Calculation

5. Your First HIP Kernel

We will implement a vector addition kernel: $C[i] = A[i] + B[i]$

This simple kernel demonstrates the basic structure of GPU programming:

1. Each thread processes one element
2. Threads compute their global index
3. Boundary checking prevents out-of-bounds access

Kernel Structure

```
__global__ void vector_add(const float* A, const float* B, float* C, int N) {
    // Step 1: Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x; // Fill in

    // Step 2: Boundary check (important when N is not divisible by block size)
    if (idx < N) { // Fill in the condition
        // Step 3: Perform computation
        C[idx] = A[idx] + B[idx]; // Fill in
    }
}
```

图 2: Exercise 2: Implement the Kernel

Exercise 3: Launch Configuration Calculation

Given $N = 1000$ elements, complete the table:

Block Size	Grid Size Formula	Grid Size	Total Threads	Idle Threads
64	$\text{ceil}(1000/64)$	16	1024	24
128	$\text{ceil}(1000/128)$	8	1024	24
256	$\text{ceil}(1000/256)$	4	1024	24
512	$\text{ceil}(1000/512)$	2	1024	24

Question: Which block size gives the best thread utilization? Why might this not always be the best choice?

Your answer: 所有列出的块大小都会产生相同的线程利用率，因为每个块都会为1000个元素启动总共1024个线程，剩余24个空闲线程。因为性能还取决于占用率、波前对齐、寄存器使用、共享内存使用、内存带宽以及调度开销。

图 3: Exercise 3: Launch Configuration Calculation

Exercise 4: Analyze Your Results

Based on the multi-trial results above:

1. Which block size achieved the **lowest mean execution time**? **256**
2. Which block size achieved the **highest memory bandwidth**? **256**
3. Are the differences between block sizes **statistically significant** (look at error bars)? **不显著，大多数误差条 (error bars) 有重叠，因此差异没有统计学意义。**
4. Why does Vector Add performance depend more on bandwidth than on block size?

Your answer: 因为 **Vector Add** 每个元素只进行一次加法运算，但需要读取多个内存数据，属于典型的内存带宽受限 (*memory-bound*) 问题，因此性能主要由内存带宽决定，而不是 *block size*。

图 4: Exercise 4: Analyze Your Results

Exercise 5: Wavefront Analysis

Question 1: A kernel is launched with block size 100. How many wavefronts does each block require? How many lanes are wasted?

- Wavefronts per block: **4** (hint: $\lceil 100/32 \rceil$)
- Wasted lanes: **28** (hint: wavefronts $\times$ 32 - 100)

Question 2: If the same kernel were compiled with `-mwavefrontsize64`, how would the answers change?

- Wavefronts per block (wave64): **2** (hint: $\lceil 100/64 \rceil$)
- Wasted lanes (wave64): **28**

Question 3: What happens when threads within a wavefront take different branches (if-else)?

Your answer: 采用不同的分支时，GPU会同时串行执行两个分支路径，而这会降低并行效率。

图 5: Exercise 5: Wavefront Analysis

Exercise 6: Occupancy Calculation

For Radeon 8060S (RDNA 3.5): 20 CUs, Warp Size = 32, Max Blocks per CU = 64, Max Threads per Block = 1024

To calculate occupancy, you need to know the max wavefronts per CU (check `rocminfo` output).

Block Size	Wavefronts/Block	Blocks that fit	Active Wavefronts	Notes
32	1	limited by max blocks (64)	64	<i>maximum active wavefronts reached</i>
64	2	32	64	<i>maximum active wavefronts reached</i>
128	4	16	64	<i>maximum active wavefronts reached</i>
256	8	8	64	<i>maximum active wavefronts reached</i>
512	16	4	64	<i>maximum active wavefronts reached</i>
1024	32	2	64	<i>maximum active wavefronts reached</i>

Note: Use `rocminfo` output from Exercise 1 to determine actual hardware limits for your device.

图 6: Exercise 6: Occupancy Calculation

13. Challenge Exercise

Modify the kernel to process **multiple elements per thread**. This is a common technique when N is very large.

```
__global__ void vector_add_multi(const float* A, const float* B, float* C,
                                int N, int elements_per_thread) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * elements_per_thread; // Fill in: total number of threads

    for (int i = tid; i < N; i += stride) {
        C[i] = A[i] + B[i];
    }
}
```

Question: What is the advantage of this approach over the 1:1 thread-to-element mapping?

这允许每个线程处理多个元素，因此内核可以处理非常大的数组，而无需为每个元素启动一个线程。

图 7: 13. Challenge Exercise

2.2 Lab2 Matrix Transpose

2.2.1 学习内容

Lab 2 主要通过矩阵转置实验，让我们理解 GPU 上二维线程的组织方式以及索引映射关系，学习如何将线程的 (blockIdx, threadIdx) 转换为全局坐标，并正确访问输入和输出矩阵中的元素。同时，实验重点分析了 GPU 内存访问模式（特别是 coalesced memory access）对性能的影响，以及为什么普通转置会产生访存效率问题。进一步通过使用 shared memory 的 tiled transpose 方法，学习如何减少全局内存访问次数并避免 bank conflict，从而显著提升性能。

2.2.2 题目截图

Exercise 1: 2D Indexing

Given `blockDim = (32, 32)`, calculate the global indices:

blockIdx	threadIdx	Global (idx, idy)
(0, 0)	(5, 10)	(5, 10)
(1, 0)	(0, 0)	(32, 0)
(2, 3)	(15, 20)	(79, 116)

Question: For a matrix with `rows=100, cols=200`, how many blocks are needed?

Your calculation: **28**

图 8: Exercise 1: 2D Indexing

Exercise 2: Understand the Index Transformation

For a 4×6 matrix (`rows=4, cols=6`), fill in the table:

Thread (idx, idy)	Input Index	Output Index
(0, 0)	$0*6+0 = 0$	$0*4+0 = 0$
(1, 0)	1	4
(0, 1)	6	1
(3, 2)	15	14

Question: Why is the write index formula `idx * rows + idy` instead of `idx * cols + idy`?

Your answer: **因为输出矩阵是转置后的结果，它的内存布局发生了变化**

图 9: Exercise 2: Understand the Index Transformation

Question: The large test has ~500x more elements than test 2. Is the time 500x longer? Why or why not?

Your answer: **不会。因为 GPU 性能不只线性取决于数据量。**

图 10: Exercise 3: Performance Analysis

Exercise 4: Coalescing Analysis

For a 1024×1024 matrix with 32×32 thread blocks (one wave32 = one row of the block):

Question 1: When reading `input[idx * cols + idx]`, are consecutive threads (in `threadIdx.x`) reading consecutive addresses?

Your answer: 是的, `threadIdx.x` 相邻线程访问连续内存地址, 因此读取是合并访问 (*coalesced access*)。

Question 2: When writing `output[idx * rows + idx]`, what is the stride between addresses written by consecutive threads?

Your answer: 相邻线程写入地址的步长 (*stride*) 是 `rows`。

Question 3: If each memory transaction fetches 128 bytes (32 floats), how many transactions are needed for 32 threads (one wavefront) to write non-coalesced?

Your answer: 需要 32 次 *memory transactions* (每个线程一次)。

图 11: Exercise 4: Coalescing Analysis

Exercise 5: Tiled Transpose Analysis

Question 1: Why do we add `+1` to the shared memory declaration (`tile[BLOCK_SIZE][BLOCK_SIZE + 1]`)?

Your answer: 为了避免 *shared memory bank conflict* (银行冲突)。

Question 2: In the write step, why do we swap `blockIdx.x` and `blockIdx.y`?

Your answer: 为了让转置后的 *tile* 在输出矩阵中正确排列, 并提高写入的内存连续性 (*coalescing*)。

Question 3: How much shared memory does each block use for `BLOCK_SIZE=32`?

Your calculation: $32 \times 33 \times 4 \text{ bytes} = 4224 \text{ bytes}$

图 12: Exercise 5: Tiled Transpose Analysis

2.3 Lab3 Histogram

2.3.1 学习内容

本次 Lab 3 主要通过实现和优化 GPU histogram 统计, 学习了并行计算中“全局内存竞争 (atomic contention)”对性能的影响。首先分析了 naive GPU 实现中每个输入元素都会触发 atomic 操作, 从而导致严重的内存冲突和性能瓶颈; 随后引入 shared memory 进行分块局部统计, 再通过 merge step 合并结果, 从而显著减少 global atomic 次数并提升性能。此外, 还通过不同数据分布实验, 理解了输入数据均匀性对 contention 和执行效率的影响。

2.3.2 题目截图

Exercise 1: Analyze the Problem

Question 1: How many times is each input element read from global memory?

Your answer: *每个 input element 只被 global memory 读取 1 次*

Question 2: If num_bins = 256 and N = 10M, what is the total global memory traffic?

Your calculation: *20,000,000 global memory operations*

Question 3: Why is this approach inefficient?

Your answer: *因为每个元素都需要进行 global memory atomic 操作, 导致严重的 memory contention 和 serialization, 无法并行化。*

图 13: Exercise 1: Analyze the Problem

Exercise 2: Understand the Optimization

Question 1: Why is atomicAdd on shared memory faster than on global memory?

Your answer: *shared memory latency 远低于 global memory, 并且 contention 发生在 CU 内部, 不需要跨 GPU 节点通信。*

Question 2: In the merge step, how many global atomicAdd operations occur per bin?

Your answer: *每个 bin 在 merge 阶段会被每个 block 更新一次*

Question 3: What is the purpose of the grid-stride loop pattern `for (int i = idx; i < N; i += totalthread) ?`

Your answer: *让线程可以跨多个元素处理数据, 提高负载均衡并充分利用 GPU 线程资源。*

图 14: Exercise 2: Understand the Optimization

Exercise 3: Record Results

Fill in the execution times:

Implementation	Time (testcase 2)	Time (testcase 4)
Serial (CPU)	18	720
Naive GPU	3.2	14.5
Optimized GPU	0.6	2.8

Calculate speedups:

- Optimized vs Serial: **257x**
- Optimized vs Naive: **5.2x**

图 15: Exercise 3: Record Results

Exercise 4: Memory Efficiency

Question: Calculate the memory traffic reduction ratio:

$$\text{Reduction} = \frac{\text{Naive reads}}{\text{Optimized reads}} = \frac{N \times \text{num_bins}}{N} = ?$$

Your answer: **256x reduction**

图 16: Exercise 4: Memory Efficiency

Exercise 5: Contention Scenarios

Scenario 1: All input values are the same (e.g., all zeros)

- What happens to performance? **所有线程写同一个 bin → atomic contention 极高**
- Why? **所有 atomicAdd 都竞争同一个 memory location**

Scenario 2: Input values are uniformly distributed across all bins

- Expected contention level: **低 contention, 因为不同线程更新不同 bin**
- Why? **256 bins 足够分散写入冲突**

图 17: Exercise 5: Contention Scenarios

2.4 Lab4 Reduction

2.4.1 学习内容

本次 Lab 4 主要学习 GPU 上的并行归约 (reduction) 算法, 包括树形归约的基本执行过程、block 内分阶段合并的实现方式, 以及 __syncthreads() 在同步线程中的作用。同时分析了 naive atomic 版本与优化版本在全局原子操作次数上的差异, 并理解 shared memory reduction 和 warp shuffle 如何减少同步开销、提高带宽利用率。最后通过 occupancy 与 contention 分析, 理解 block size、warp 利用率与性能之间的权衡关系。

2.4.2 题目截图

Exercise 1: Understand Tree Reduction

Question 1: For an array of 1024 elements with block size 256, how many reduction steps occur within each block?

Your answer: **8 steps**

Question 2: After block-level reduction, how many partial sums remain?

Your answer: **4 partial sums**

Question 3: Why do we use `__syncthreads()` after each reduction step?

Your answer: **保证同一个 block 内所有线程在进入下一轮 reduction 前完成当前计算，避免数据竞争和未定义行为。**

图 18: Exercise 1: Understand Tree Reduction

Exercise 2: Analyze Occupancy Results ¶

From the test output, fill in the table:

Block Size	Warps/Block	Occupancy (%)	Execution Time (ms)
64	2	25%	0.42
128	4	50%	0.30
256	8	100%	0.22
512	16	100%	0.24
1024	32	100%	0.28

Question: Does higher occupancy always mean better performance?

Your answer: **不一定。因为性能还受 memory bandwidth、register pressure 和 synchronization overhead 影响。**

图 19: Exercise 2: Analyze Occupancy Results

Exercise 3: Think About Trade-offs

Question 1: With block size 64, how many final atomic operations occur for $N = 1\text{M}$ elements?

Calculation: $\text{grid_size} = \text{ceil}(1,000,000 / 64) = \mathbf{15625}$

Question 2: With block size 1024, how many final atomic operations occur?

Calculation: $\text{grid_size} = \text{ceil}(1,000,000 / 1024) = \mathbf{977}$

Question 3: Why might fewer atomic operations improve performance?

Your answer: **因为 atomic 操作具有 serialization 特性，会导致线程竞争，降低并行效率。**

图 20: Exercise 3: Think About Trade-offs

Exercise 4: Algorithm Analysis

Question 1: In tree reduction, what fraction of threads are idle after the first step?

Your answer: **$1/2$ threads idle**

Question 2: Why does warp shuffle not require `__syncthreads()` ?

Your answer: **因为 warp shuffle 在 warp 内部进行寄存器级通信，不涉及 shared memory，因此天然同步。**

Question 3: How does multi-element per thread improve memory bandwidth utilization?

Your answer: **减少线程调度开销，提高 memory throughput，并提升 GPU 资源利用率。**

图 21: Exercise 4: Algorithm Analysis

2.5 Lab5 Monte Carlo Integration

2.5.1 学习内容

本次 Lab 5 主要学习 GPU 上 Monte Carlo / 数值积分算法的并行实现方式，重点分析了 atomic reduction 的开销、grid-stride loop 的作用以及共享内存优化思想。通过比较 naive atomic 版本与优化版本，理解了全局原子操作的性能瓶颈，并学习如何通过“每线程局部累加 + 最终归约”减少 atomic 竞争。同时结合数值积分问题，理解了并行计算中“重复计算 vs 同步开销”的权衡关系。

2.5.2 题目截图

Exercise 1: Understand the Algorithm

Question 1: Why is the division by `n_samples` done in each thread instead of once at the end?

Your answer: 因为每个线程独立计算部分结果, 如果不提前归一化, 会导致最后需要额外同步累加, 增加 `atomic` 开销和误差传播。

Question 2: What is the time complexity of the atomic reduction approach?

Your answer: $O(N)$

Question 3: How could you modify this kernel to use shared memory reduction instead of atomics?

Your answer: 每个 `block` 先在 `shared memory` 中做局部 `reduction`, 然后再进行一次 `global atomic update`。

图 22: Exercise 1: Understand the Algorithm

Exercise 2: Manual Calculation

For Test 1, verify the result manually:

Given: $a = 0$, $b = 2$, $n = 8$ samples

$$\text{Formula: result} = \frac{(b-a)}{n} \sum_{i=1}^n y_i = \frac{2}{8} \times \sum y_i$$

Your calculation:

- Sum of y values: **9.0**
- Expected result: **2.25**
- Does it match the program output? **Yes**

图 23: Exercise 2: Manual Calculation

Question 1: When sample count increases 100x (from test 4 to 7), how much does execution time increase?

Your answer: 时间不会严格增加100倍, 通常略低于100倍, 因为GPU调度和cache效率会更好。

Question 2: Is the scaling linear? Why or why not?

Your answer: 不是完全线性。

图 24: Exercise 3: Scalability Analysis

Exercise 4: Bottleneck Analysis

Question 1: If the computation takes 1 cycle and atomic takes 100 cycles, what is the theoretical efficiency for N=1M threads?

Hint: Total work = N x (compute + atomic)

Your calculation: *≈ 0.99% efficiency*

Question 2: How would shared memory reduction improve this?

Your answer: *通过减少 global atomic 次数, 将 100-cycle atomic 转换为 shared memory reduction, 大幅降低延迟。*

Question 3: With 256 threads per block and N=1M, how many global atomics would shared memory reduction require?

Your calculation: *3906 global atomic operations*

图 25: Exercise 4: Bottleneck Analysis

Exercise 5: Precision Considerations

Question 1: The kernel uses `double` (64-bit). How many significant digits does double precision provide?

Your answer: *double 提供约 15~16 位有效数字*

Question 2: If we switch to `float` (32-bit), what problems might occur with N=100M samples?

Your answer: *结果可能出现明显数值误差, 尤其在大规模累加时误差累积严重。*

图 26: Exercise 5: Precision Considerations

Exercise 6: Optimization Analysis

Question 1: In the optimized version, how many global atomics occur for N=1M with 256 blocks?

Your answer: *256 global atomic operations*

Question 2: What is the purpose of the grid-stride loop `for (int i = idx; i < n_samples; i += blockDim.x * gridDim.x) ?`

Your answer: *提高线程复用率, 使每个线程可以处理多个 sample, 提高 GPU 利用率。*

Question 3: Why is `(b - a) / n_samples` multiplication done only by thread 0?

Your answer: *提高线程复用率, 使每个线程可以处理多个 sample, 提高 GPU 利用率。*

图 27: Exercise 6: Optimization Analysis

2.6 Lab6 K-Means Clustering

2.6.1 学习内容

本次 Lab 6 主要学习 GPU 上 K-Means 聚类算法的并行实现方式, 包括 assignment (分配最近中心点) 和 recalculation (更新 centroid) 两个核心 kernel。重点分析了 global atomic 在更新簇统计时的性能瓶颈, 以及 shared memory reduction 如何减少原子操作开销。同时学习了 grid-stride loop 提高线

程利用率、centroid 缓存优化访存效率，以及 convergence 控制和 double buffering 技术在迭代算法中的作用。

2.6.2 题目截图

Exercise 1: Algorithm Analysis ¶

Question 1: In the assignment kernel, what is the time complexity per thread if $k = 100$?

Your answer: $O(k)$

Question 2: Why does the recalculation kernel use shared memory atomics before global atomics?

Your answer: 因为 *shared memory atomic* 比 *global atomic* 快很多，可以先在 *block* 内聚合，再减少 *global atomic* 次数，从而降低全局竞争。

Question 3: What happens if a centroid has no points assigned to it (empty cluster)?

Your answer: 该 *centroid* 不会被更新，通常保持旧值或需要重新初始化。

图 28: Exercise 1: Algorithm Analysis

Exercise 2: Performance Analysis

Record the execution time for test 4:

- Sample size: 50,000
- Clusters (k): 50
- Max iterations: 100
- Execution time: **0.85ms**

Question: What is the dominant computation in each iteration?

Your answer: ***distance computation*** (距离计算) 占主导

图 29: Exercise 2: Performance Analysis

Exercise 3: Compute Intensity ¶

For $N = 50,000$ points and $k = 50$ clusters:

Assignment kernel:

- Distance calculations per point: **50**
- Total distance calculations: **$50,000 \times 50 = 2,500,000$**

Update kernel:

- Shared memory atomics: **50,000**
- Global atomics (with 256 threads/block): **196**

图 30: Exercise 3: Compute Intensity

Exercise 4: Memory Optimization

Question 1: The centroids are read by every thread. How could you optimize this access pattern?

Your answer: 将 *centroid* 缓存在 *shared memory* 或 *register* 中, 减少重复 *global memory* 访问, 并提升 *cache hit rate*.

Question 2: If $k = 1000$, what would be the impact on cache performance?

Your answer: *centroids* 过大无法放入 *cache*, 会导致 *cache miss* 增加, *memory bandwidth* 压力显著上升。

图 31: Exercise 4: Memory Optimization

Exercise 5: Convergence Analysis

Question 1: What is the convergence threshold in this implementation?

Your answer: ***epsilon = 0.001***

Question 2: Why is it important to check convergence rather than always running max_iterations?

Your answer: **因为大多数情况下算法会提前收敛，继续迭代只会浪费计算资源。**

Question 3: How does the double-buffering (swapping prev and new centroids) help?

Your answer: **避免读写冲突，使 old centroids 和 new centroids 分离，防止数据覆盖并提升并行效率。**

图 32: Exercise 5: Convergence Analysis

3 口令生成加速 (CUDA)

3.1 实现思路

本次 CUDA 加速采用“CPU 调度 + GPU 批量展开”的混合并行策略。整体流程如下：

1. CPU 端继续执行 PCFG 模型训练、概率排序、优先队列维护和 PT 出队操作；
2. 当 `PriorityQueue::Generate(PT pt)` 被调用时，CPU 端先判断当前 PT 的 segment 数量；
3. 若 PT 只有一个 segment，则该 segment 的所有 `ordered_values` 均可直接作为候选口令，GPU 对这些 value 进行批量输出；
4. 若 PT 包含多个 segment，则 CPU 先根据 `curr_indices` 生成除最后一个 segment 外的固定前缀 `prefix`；
5. GPU 负责并行执行 `prefix + ordered_values[i]` 的拼接操作，其中一个 CUDA 线程负责生成一个候选口令；
6. GPU 生成完成后，结果拷贝回 CPU 端，并追加到 `PriorityQueue::guesses` 中，供后续哈希计算阶段使用。

该设计的关键思想是将规则、稳定、可批量展开的字符串生成部分交给 GPU，而将不规则、依赖优先队列和动态结构的控制逻辑保留在 CPU 端。这样既避免了在 GPU 上实现复杂优先队列，也能利用 GPU 大量线程对候选口令生成阶段进行并行加速。

在 CUDA kernel 中，采用 one-thread-per-candidate 的并行模型。设当前需要展开的 value 数量为 N ，则启动不少于 N 个 CUDA 线程。第 i 个线程读取第 i 个 value，并将其与公共前缀 `prefix` 拼接，生成第 i 个候选口令。由于不同线程之间不共享输出位置，因此不需要复杂的线程同步。

3.2 代码实现

本次实现中，在原有宏控制结构中加入 CUDA_PARALLEL 宏，用于切换到 GPU 加速版本。宏控制部分如下：

```

1 // Guess generation parallel control
2 //define SERIAL
3 //define OPENMP
4 //define PTHREAD
5 //define MPI_PARALLEL
6 #define CUDA_PARALLEL

```

在 guessing.cpp 中，原有 PriorityQueue::Generate(PT pt) 函数保留整体结构，只在两个主要分支中加入 CUDA 版本代码。对于只有一个 segment 的情况，直接调用 CUDA 批量生成函数：

```

1 #ifdef CUDA_PARALLEL
2     segment *a;
3     if (pt.content[0].type == 1)
4         a = &m.letters[m.FindLetter(pt.content[0])];
5     else if (pt.content[0].type == 2)
6         a = &m.digits[m.FindDigit(pt.content[0])];
7     else
8         a = &m.symbols[m.FindSymbol(pt.content[0])];
9
10    CUDA_GenerateBatch(
11        guesses,
12        "",
13        a,
14        pt.max_indices[0]
15    );
16 #endif

```

其中，空字符串 "" 表示当前候选口令没有前缀，GPU 只需要将 segment 中的每个 value 直接写入输出结果。

对于多个 segment 的情况，CPU 首先根据当前 PT 的 curr_indices 拼接除最后一个 segment 以外的前缀，然后将该前缀和最后一个 segment 的 value 列表传给 GPU：

```

1 #ifdef CUDA_PARALLEL
2     string prefix;
3     int seg_idx = 0;
4
5     for (int idx : pt.curr_indices)
6     {
7         if (pt.content[seg_idx].type == 1)
8             prefix += m.letters[m.FindLetter(pt.content[seg_idx]).ordered_values[idx];
9         else if (pt.content[seg_idx].type == 2)
10            prefix += m.digits[m.FindDigit(pt.content[seg_idx]).ordered_values[idx];
11        else

```

```

12     prefix += m.symbols[m.FindSymbol(pt.content[seg_idx])].ordered_values[idx];
13
14     seg_idx += 1;
15
16     if (seg_idx == pt.content.size() - 1)
17         break;
18 }
19
20 segment *a;
21 int last = pt.content.size() - 1;
22
23 if (pt.content[last].type == 1)
24     a = &m.letters[m.FindLetter(pt.content[last])];
25 else if (pt.content[last].type == 2)
26     a = &m.digits[m.FindDigit(pt.content[last])];
27 else
28     a = &m.symbols[m.FindSymbol(pt.content[last])];
29
30 CUDA_GenerateBatch(
31     guesses,
32     prefix,
33     a,
34     pt.max_indices[last]
35 );
36 #endif

```

其中，变量 `guess` 表示已经在 CPU 端确定好的前缀，`a` 指向最后一个 `segment` 在模型中的统计结果，`pt.max_indices[last_idx]` 表示该 `segment` 中需要遍历的 `value` 数量。

本次 CUDA 后端主要包含两个新增文件：`guessing_cuda.cuh` 和 `guessing_cuda.cu`。

`guessing_cuda.cuh` 是 CUDA 后端的接口头文件，主要负责声明 CPU 端可调用的 CUDA 封装函数，例如：

```

1 void CUDA_Init();
2 void CUDA_Finalize();
3
4 long long CUDA_GenerateBatch(
5     std::vector<std::string> &guesses,
6     const std::string &prefix,
7     segment *seg_ptr,
8     int total
9 );

```

其中，`CUDA_GenerateBatch()` 是本次 GPU 加速的核心接口，负责将 `prefix` 与某个 `segment` 的全部 `ordered_values` 批量拼接，并将结果追加到 `guesses` 中。

`guessing_cuda.cu` 是 CUDA 后端的具体实现文件，主要包含 CUDA kernel、GPU 内存申请与释放、CPU-GPU 数据拷贝以及 kernel 启动逻辑。核心 kernel 的功能是让每个线程生成一个候选口令，其基本逻辑如下：

```

1 __global__ void cuda_expand_values_kernel(

```

```

2  const char *prefix,
3  int prefix_len,
4  const char *values,
5  int value_stride,
6  char *output,
7  int output_stride,
8  int total)
9  {
10     int idx = blockIdx.x * blockDim.x + threadIdx.x;
11
12     if (idx >= total)
13     {
14         return;
15     }
16
17     const char *value_ptr = values + idx * value_stride;
18     char *out_ptr = output + idx * output_stride;
19
20     // 1. 拷贝 prefix
21     for (int i = 0; i < prefix_len; i += 1)
22     {
23         out_ptr[i] = prefix[i];
24     }
25
26     // 2. 拼接 value
27     int j = 0;
28
29     while (j < value_stride)
30     {
31         char c = value_ptr[j];
32
33         if (c == '\0')
34         {
35             break;
36         }
37
38         out_ptr[prefix_len + j] = c;
39         j += 1;
40     }
41
42     // 3. 字符串结尾
43     out_ptr[prefix_len + j] = '\0';
44 }

```

在 CPU 封装函数 `CUDA_GenerateBatch()` 中，程序首先将 `ordered_values` 打包成连续字符数组，然后调用 `cudaMalloc()` 在 GPU 端申请显存，使用 `cudaMemcpy()` 将 `prefix` 和 `values` 传入 GPU，随后启动 kernel 并行生成候选口令。kernel 执行结束后，程序再将 GPU 输出结果拷贝回 CPU，并转换为 `std::string` 后追加到 `guesses` 容器中。

该实现的优点是对原有 PCFG 框架侵入性较低，保留了原先串行、OpenMP、Pthread 和 MPI 版本的整体组织方式，只是额外增加了 CUDA 作为一种新的并行后端。同时，该实现也便于与已有结果进行横向对比。

4 实验结果与分析

本实验对串行版本和 CUDA 版本进行了对比。测试时生成约 10^7 个候选口令，并记录模型训练时间、口令生成时间、SIMD 哈希计算时间以及最终生成的候选口令总数。实验结果如表 1 所示。

表 1: 串行版本与 CUDA 版本实验结果对比

实现方式	Guess time/s	SIMD Hash time/s	Train time/s	Total guesses generated
Serial	0.348059	1.78370	26.8450	10035780
CUDA	0.286420	1.82137	28.7645	10121436

根据表 1，CUDA 版本的口令生成时间由串行版本的 0.348059 s 降低到 0.286420 s。由此可计算口令生成阶段的加速比为：

$$S_{\text{guess}} = \frac{T_{\text{serial}}}{T_{\text{CUDA}}} = \frac{0.348059}{0.286420} \approx 1.22$$

即 CUDA 版本在口令生成阶段获得了约 $1.22\times$ 的加速。该结果说明，`Generate()` 函数中最后一层 segment value 的展开过程确实具有一定的数据并行性，适合映射到 GPU 上执行。

为了进一步分析系统整体耗时结构，可以计算不同阶段在完整运行时间中的占比。若将训练、口令生成和哈希计算三部分相加作为主要运行时间，则串行版本总时间约为：

$$T_{\text{serial,total}} = 26.8450 + 0.348059 + 1.78370 = 28.976759 \text{ s}$$

CUDA 版本总时间约为：

$$T_{\text{CUDA,total}} = 28.7645 + 0.286420 + 1.82137 = 30.87229 \text{ s}$$

可以看到，虽然 CUDA 版本在口令生成阶段取得了加速，但整体运行时间并未显著下降。这主要是因为当前实验设置下，训练阶段耗时占比最高，而本次 CUDA 加速并未作用于模型训练阶段。此外，哈希计算阶段仍采用 SIMD 实现，也没有迁移到 GPU。因此，GPU 优化的实际收益主要体现在 Guess time，而非完整程序总时间。

从阶段占比看，串行版本中 Guess time 仅占主要运行时间的约：

$$\frac{0.348059}{28.976759} \times 100\% \approx 1.20\%$$

CUDA 版本中 Guess time 占比进一步下降为：

$$\frac{0.286420}{30.87229} \times 100\% \approx 0.93\%$$

这说明在当前数据规模和程序结构下，口令生成阶段已经不是完整链路中最主要的瓶颈。真正占主要比例的是模型训练阶段和哈希计算阶段。因此，后续若希望继续提升完整系统性能，应考虑以下方向：

1. 对训练阶段进行并行化优化，例如并行统计 segment 和 preterminal 频率；
2. 将哈希计算阶段进一步迁移到 GPU，使 GPU 不仅负责候选口令生成，也负责批量哈希计算；

3. 减少 CUDA 版本中的 `cudaMalloc()`、`cudaMemcpy()` 和 `cudaFree()` 开销，通过显存复用或批处理流水线降低 CPU-GPU 数据传输成本；
4. 将多个 PT 的展开任务合并成更大的 batch，以提高 GPU kernel 的计算密度，减少小规模 kernel 启动开销；
5. 设计 CPU-GPU 异步流水线，使 CPU 在维护优先队列的同时，GPU 并行处理上一批候选口令生成任务。

5 总结

综上，本次实验实现了 PCFG 口令生成阶段的 CUDA 并行化。实验结果表明，GPU 对候选口令批量生成具有一定加速效果，口令生成阶段加速比约为 $1.22\times$ 。但由于当前程序的主要耗时集中在模型训练和哈希计算阶段，单独加速口令生成对整体运行时间的改善有限。这也说明在异构并行系统设计中，仅优化局部热点并不一定能带来整体性能的显著提升，需要结合完整链路的耗时占比进行系统级优化。